

Capítulo 19

Sobrecarga de Operadores; Classe `string`

Exercícios de Autorrevisão — Resolução Didática

“Sobrecarga de operadores é um dos recursos mais distintivos de C++. Ele nos permite escrever `c1 + c2` no lugar de `c1.add(c2)`, `cout << minhaConta << endl` no lugar de `minhaConta.print()`, e `r[5] = 10` no lugar de `r.set(5, 10)`. É o coroamento sintático do projeto de classes: o tipo definido pelo programador ganha cidadania de primeira classe na linguagem, com notação operacional idêntica à dos tipos primitivos.”

1 Exercício 19.1 — Vocabulário do Capítulo 19

Preencha os espaços em cada uma das sentenças (a)–(g).

(a) `a + b` (inteiros) e `c + d` (*floats*) usam o mesmo `+`, mas para fins diferentes. Isto é um exemplo de _____

Resposta: *sobrecarga de operador (operator overloading).*

C++ vem com o `+` já sobrecarregado *para tipos primitivos*: `int + int` faz soma inteira (rapidamente, em hardware); `double + double` faz soma em ponto flutuante (com unidade de hardware distinta); `int + double` faz *promoção* de `int` para `double` e soma em ponto flutuante. O mesmo *símbolo* (`+`) representa operações *conceitualmente análogas*, mas implementadas distintamente. C++ permite ao programador estender essa polivalência para seus próprios tipos.

(b) Palavra-chave que apresenta uma função operador sobrecarregada

Resposta: `operator`.

```
Complex Complex::operator+(const Complex &o) const {  
    //      ~~~~~  
    return Complex(parteReal + o.parteReal,  
                   parteImaginaria + o.parteImaginaria);  
}
```

O nome da função é literalmente `operator+` (sem espaço); essa palavra-chave forma uma *espécie de identificador* reconhecido pelo compilador.

(c) Operadores que não precisam ser sobrecarregados

Resposta: atribuição (=), endereço (&) e vírgula (,).

C++ aplica esses três *automaticamente* a todo tipo definido pelo programador:

- **=:** o compilador gera uma atribuição *membro-a-membro* (já vista no Cap. 17). Funciona perfeitamente para classes sem gerenciamento de recursos; mas *precisa* ser sobrecarregada para classes que contém ponteiros para memória dinâmica (alvo clássico da *Regra dos Três*).
- **&:** retorna o endereço do objeto. Raramente precisa ser sobrecarregado.
- **,:** avalia os operandos da esquerda para a direita e retorna o da direita. Raramente sobrecarregado — e quando o é, geralmente confunde mais do que ajuda.

(d) Três propriedades não alteráveis pela sobrecarga

Resposta: precedência, associatividade e aridade (número de operandos).

- **Precedência.** * sempre tem precedência maior que +. Mesmo se você sobrecarregar ambos para uma classe `Matriz`, em `m1 + m2 * m3` a multiplicação ocorre primeiro — você não pode alterar isso.
- **Associatividade.** `a = b = c` é `a = (b = c)` (direita para esquerda); `a + b + c` é `(a + b) + c` (esquerda para direita). Também imutável.
- **Aridade.** + sempre aceita dois operandos quando binário, um quando unário (+x). Você não pode definir um `operator+` de três operandos.

Observação de Engenharia de Software

Essas três restrições preservam a *previsibilidade* sintática de C++. O programador pode olhar uma expressão e saber a ordem de avaliação apenas pelas regras universais, sem precisar consultar a definição das classes envolvidas. Sem essa garantia, ler código C++ seria virtualmente impossível.

(e) Operadores que *não* podem ser sobrecarregados

Resposta: `.`, `?:`, `.*` e `::`.

Os cinco operadores “sagrados” de C++:

- **.** (acesso a membro): sobrecarregá-lo destruiria toda a sintaxe de acesso a membros.
- **?:** (operador ternário condicional): único ternário; permitir sobrecarga complicaria gravemente o analisador.
- **.*** (acesso a membro via ponteiro-para-membro): raro mas necessário para certas semânticas.
- **::** (resolução de escopo): em `std::cout`, *precisa* ter significado estático, não pode depender do tipo à esquerda.
- **sizeof** (mencionado no livro como “operador”): também não sobrecarregável.

(f) Operador que libera memória alocada por new

Resposta: delete.

Para um objeto: `delete p;`. Para um array: `delete[] p;`. Usar a forma errada, ou esquecer-se de delete, são as duas principais fontes de vazamentos de memória em C++.

(g) Operador que aloca memória dinamicamente e retorna _____

Resposta: new; retorna um ponteiro para o tipo solicitado.

```
int    *p = new int(42);           // ponteiro para int alocado
      dinamicamente
Time   *t = new Time(12, 0, 0);    // ponteiro para objeto Time
int    *v = new int[100];          // ponteiro para array de 100 ints
```

Boa Prática de Programação

Em C++ moderno (C++11+), evite new/delete diretos sempre que possível. Use:

- `std::vector<T>` em vez de `new T[N]`.
- `std::unique_ptr<T>` em vez de `new T()` com responsabilidade única de delete.
- `std::shared_ptr<T>` para propriedade compartilhada.

Essas abstrações eliminam quase todos os vazamentos. Mas para os exercícios deste capítulo, new/delete crus são usados para fins didáticos.

2 Exercício 19.2 — Significados de « e »

1. « significa:

- **Deslocamento de bits à esquerda**, quando aplicado a inteiros: `5 « 2` produz 20.
- **Inserção de stream**, quando aplicado a ostream (como cout): `cout « "01a"`.

2. » significa:

- **Deslocamento de bits à direita**, quando aplicado a inteiros: `20 » 2` produz 5.
- **Extração de stream**, quando aplicado a istream (como cin): `cin » x`.

A diferenciação depende do **contexto** — mais especificamente, dos tipos dos operandos. C++ *escolhe* a interpretação adequada com base nas regras de sobrecarga. `int « int` dispara o significado bit-a-bit; `ostream « X` (para qualquer X) dispara a inserção.

Observação de Engenharia de Software

Esse caso é particularmente interessante: a biblioteca-padrão *aproveitou* a sobrecarga para reinterpretar *semanticamente* «. Em sistemas Unix, « no shell significa *here-document* (`cat « EOF`); em C++, foi reciclado para *streaming*. A escolha foi mnemonica: visualmente, « "01a" “empurra” o texto para dentro de cout; » x “puxa” do cin para x.

3 Exercício 19.3 — Em que contexto `operator/` pode aparecer?

Em sobrecarga de operador, `operator/` é o nome de uma função que oferece a versão sobrecarregada do operador `/` (divisão) para uma classe específica.

Exemplo

```
class Rational {
public:
    Rational operator/((const Rational &o) const {
        return Rational(num * o.den, den * o.num);
    }
private:
    int num, den;
};

// Uso:
Rational r1(1, 2), r2(1, 3);
Rational q = r1 / r2;          // q = 3/2 (chama r1.operator/(r2))
```

4 Exercício 19.4 — V/F: somente operadores existentes podem ser sobrecarregados

Resposta: Verdadeiro.

Você não pode *inventar* um novo operador. Tentativas como `operator&&&` (três `&`s) ou `operator**` (potenciação, comum em Fortran e Python) não compilam. C++ permite reusar símbolos *já definidos pela linguagem*, mas não acrescentar novos símbolos ao léxico.

Por que essa restrição?

Observação de Engenharia de Software

Permitir operadores customizados (como em algumas linguagens funcionais: Haskell, Scala) tornaria o *lexer* de C++ ambigualmente variável: o significado de cada caractere `$`, `?`, `@` dependeria do contexto. C++ optou pela conservação — o conjunto de operadores é *fixo*, conhecido antecipadamente pelo compilador, e estende-se apenas o *significado* para tipos definidos pelo usuário.

5 Exercício 19.5 — Precedência de operador sobrecarregado

Qual é a diferença entre a precedência de um operador sobrecarregado e a precedência do operador original?

A precedência é idêntica. A sobrecarga não pode alterar a precedência do operador.

Consequência prática

Em `m1 + m2 * m3` para matrizes (com `operator+` e `operator*` ambos sobrecarregados), C++ avalia *primeiro* `m2 * m3`, *depois* `m1 + (resultado)` — exatamente como faria com inteiros. Isso reforça a *intercambialidade conceitual* dos tipos primitivos com os definidos pelo usuário.

Como mudar a ordem?

Se você *precisa* que a soma aconteça primeiro, use parênteses — como em qualquer expressão matemática:

```
(m1 + m2) * m3           // soma primeiro, depois multiplica
```

Boa Prática de Programação

Por essa razão, ao sobrecarregar operadores, *respeite a semântica natural* de cada um. Sobrecarregar `+` para fazer subtração seria diabolicamente surpreendente; sobrecarregar `«` para fazer multiplicação seria pior ainda. Os operadores devem manter consistência com a operação matemática que tradicionalmente representam.

Síntese conceitual

Os cinco exercícios de autorrevisão concentram os pontos-chave da sobrecarga de operadores em C++:

- **Conceito** (19.1a): a sobrecarga é a possibilidade de associar múltiplas implementações ao mesmo símbolo.
- **Sintaxe** (19.1b): a palavra-chave `operator` introduz a definição.
- **Restrições intransponíveis** (19.1c-e, 19.4, 19.5): precedência, associatividade, aridade e o conjunto de símbolos são fixos.
- **Operadores especiais** (19.1f-g): `new/delete` para gerenciamento dinâmico, fundamento para classes que contêm ponteiros.
- **Significado contextual** (19.2): `«` e `»` mudam de comportamento conforme os operandos — exemplo clássico de polimorfismo estático.

Esses cinco pontos formam a teoria; os exercícios 19.6 a 19.11 dão a prática.